Higher Diploma in Software Engineering

ITS 2130

Enterprise Cloud Architecture

Final Project Guidelines - v1.0.0

Table of Contents

Table of Contents 1

1. Introduction 2

2. Capstone Project Requirement (Mandatory First Step) 3

3. Project Architecture Requirements 4

    Technology Requirements 4

4. Database Requirements (Mandatory) 5

5. Microservice Platform Components 6

    Required Platform Services 6

6. Repository Structure (Polyrepo with Git Submodules) 7

7. Frontend Application 8

8. Cloud Deployment Requirements 9

9. Cloud Storage Requirement (Mandatory) 10

10. High Availability and Auto Scaling Requirements (Mandatory) 11

    Microservice Auto Scaling 11

    Platform High Availability 11

11. Process Management and Automatic Restart Requirement (Mandatory) 12

    PM2 Configuration Expectations 12

    Mandatory Reliability Requirement 12

12. Mandatory Requirement: Expose Eureka Dashboard 13

13. Screen Recording Requirement 14

    VM Verification (Mandatory) 14

14. Project Submission Deadline 15

15. Alternative Option (For Busy Students) 16

16. Final Note 17

17. Changelog 18

1. Introduction

This document describes the requirements for the final project of the Enterprise Cloud Architecture module.

The objective of this project is to evaluate your ability to design, develop, and deploy a **cloud-native microservice-based application** using the technologies and architectural patterns discussed during this module.

Students are expected to:

- Design a microservice-based system
- Implement the services using Spring Boot
- Deploy the system to the cloud (GCP)
- Demonstrate the cloud infrastructure used to run the system

The final project must be submitted **on or before 22nd March (Week 8)**.

Late submissions **will not be accepted**.

2. Capstone Project Requirement (Mandatory First Step)

Before starting your own project, you must first ensure that you can successfully deploy the **Capstone Project** as a **cloud-native application**.

All steps have already been demonstrated during the lectures where we deployed the system to **Google Cloud Platform (GCP)**.

When developing your own project, you should follow **the same architecture and deployment process** used in the capstone project.

If you still have questions about the platform components or deployment process, please contact me through the **Slack channel**.

3. Project Architecture Requirements

Your project must follow a **Microservice Architecture**.

- Minimum number of microservices: **2**
- Maximum number of microservices: **5**

Because of the limited time available for this module, it is **strongly recommended to build 3 microservices**, which is considered the ideal number for this project.

All microservices are homogenous (must use the **same technology stack)**.

Technology Requirements

All microservices must be developed using:

- **Java 25**
- **Spring Boot (latest version)**
- **Spring Cloud**
- **Spring Data**

4. Database Requirements (Mandatory)

Your system must demonstrate the use of **both relational and non-relational databases**.

This requirement is **mandatory and non-negotiable**.

At minimum:

One microservice must use a **Relational Database**

- MySQL, PostgreSQL

Another microservice must use a **Non-Relational Database**

- MongoDB

Both database types must be included in your system.

5. Microservice Platform Components

Your microservice platform must include the following components using **Spring Cloud**.

Required Platform Services

Your architecture must include:

1. **Config Server**

   Used to centralize and externalize configuration for all microservices.
2. **Eureka Service Registry**

   Used for service registration and service discovery.
3. **API Gateway**

   Used as the single entry point for all backend services.

These components must be implemented using **Spring Cloud projects**.

6. Repository Structure (Polyrepo with Git Submodules)

Your project must follow a **polyrepo architecture**.

Each microservice and platform components must be stored in its **own GitHub repository**.

However, for submission purposes you must create **one main GitHub repository**.

This repository must contain all other repositories as **Git submodules**.

From this main repository, I must be able to navigate to all repositories related to your project.

This **main repository will be your final submission**.

7. Frontend Application

Your project must also include a **Frontend Application**.

You may use **any frontend technology**.

The frontend must properly **consume the backend APIs** deployed on **Google Cloud Platform**.

The purpose of the frontend is to demonstrate that your microservices are functioning correctly. Your frontend does not need to be elegant, polished, or visually impressive. UI design will not be evaluated in this project. The only requirement is that the frontend successfully consumes all the backend services that you created.

In other words, your frontend must demonstrate that all microservices are accessible and functioning correctly through the API Gateway.

A simple interface is completely acceptable as long as it interacts correctly with your backend services.

8. Cloud Deployment Requirements

Your system must be deployed to **Google Cloud Platform (GCP)**.

Your project must demonstrate the use of the following cloud infrastructure:

- VM Instance Groups
- Virtual Machines (VMs)
- VM Instance Templates
- Disk Images
- Health Checks
- Cloud DNS
- Load Balancing
- Cloud NAT Gateway
- Cloud SQL Instances
- Firestore
- Cloud Storage (Buckets)
- Cloud Router
- VPC Network
- Firewall Rules

These resources must be visible in your **GCP Console**.

9. Cloud Storage Requirement (Mandatory)

Your system must demonstrate the use of **Google Cloud Storage Buckets**.

At least **one microservice must store files inside a Cloud Storage bucket**.

This functionality must be visible through your **API or frontend application**.

10. High Availability and Auto Scaling Requirements (Mandatory)

Your system must demonstrate **cloud-native scalability and high availability**.

This requirement is **mandatory and non-negotiable**.

Microservice Auto Scaling

- All backend microservices must support **automatic horizontal scaling**.
- Your services must be deployed using infrastructure capable of **auto-scaling**.
- Microservices must **not run as single fixed instances**.

Platform High Availability

- The **platform layer** must run with **multiple instances**.
- These services must run **multiple instances distributed across different zones** in the region.

This ensures:

- High availability
- Fault tolerance
- Service continuity even if a zone fails

Single-instance deployments of these services **will result in mark deductions**.

11. Process Management and Automatic Restart Requirement (Mandatory)

As explained in the lectures, these applications are **not containerized**.

Therefore you must use **PM2** to manage your application processes.

This requirement is **mandatory and non-negotiable**.

PM2 Configuration Expectations

Your PM2 setup must ensure that:

- Applications are started and managed by PM2
- Logs are written to the correct location
- Applications automatically restart if a failure occurs
- PM2 starts automatically when the VM restarts

Mandatory Reliability Requirement

Automatic restart on failure is a **mandatory feature of this project**.

Your deployment must demonstrate **process-level fault tolerance**.

If services stop unexpectedly, they must **restart automatically without manual intervention**.

Improper process management will result in **mark deductions**.

12. Mandatory Requirement: Expose Eureka Dashboard

You must expose the **Eureka Service Registry Web Interface**.

This is **mandatory**.

Your GitHub **README.md** file must include the public URL of the **Eureka Dashboard**

If this link is missing, **marks will be deducted**.

13. Screen Recording Requirement

You must submit a **short screen recording** showing your deployed cloud infrastructure.

- **The recording must include navigation through all GCP resources mentioned above.**
- **VM Verification (Mandatory)**

In addition to showing the cloud infrastructure, you must also verify that your services are running on the virtual machines.

For each VM used in your project:

1. SSH into the VM from the GCP Console
2. Execute the command: `pm2 monit`
3. Display the running processes managed by PM2

This must clearly show that your application services are running and being managed by PM2.

Important notes:

- **Voice recording is not required**
- Simply record your screen while navigating the GCP Console
- All **IP addresses shown in GCP must match your project configuration**.
- If the IP addresses do not match your deployed project, it may be considered **someone else's work**, and **no marks will be awarded**.

14. Project Submission Deadline

Submission deadline:

22nd March

Projects submitted **after this date will not be accepted**.

Please plan your work accordingly.

15. Alternative Option (For Busy Students)

For students who have **very limited time**, an alternative option is available.

You may submit the **Capstone Project** as your final project.

If you submit the Capstone Project:

Maximum marks you can obtain: **60**

This means:

- You **can pass the module**
- But you **cannot obtain full marks**

If you want to achieve **higher marks**, you must submit **your own original project**.

16. Final Note

This alternative option is provided so that **every student has a fair opportunity to pass the module**.

Even if you are very busy, simply reviewing the lectures and deploying the **Capstone Project** will allow you to complete this module successfully.

If someone still fails the module after this opportunity, there is nothing more I can do.

17. Changelog

All notable changes to this module will be documented here.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

Mar 10, 2026- [v1.0.0]

- Initial release